

Examen blanc — Architecture & Exploitation Cloud Azure

Format : devoir sur table, 2 h, sans document ni ordinateur. **Barème : /40.** Mise en situation + questions de choix d'architecture et techniques. Un **corrigé indicatif** se trouve en seconde partie — essaie d'abord de répondre seul, puis compare.

Énoncé — Cas « MediTrack »

Conseil méthode : lis tout l'énoncé avant de répondre. Pour chaque choix, **justifie** (un choix sans justification ne rapporte pas les points). Une réponse « ça dépend » est valable si tu expliques de quoi ça dépend.

Contexte. *MediTrack* est une PME qui édite une application web de gestion de cabinets médicaux : prise de rendez-vous, dossiers patients, dépôt de documents (ordonnances, comptes-rendus en PDF) et facturation. L'application est utilisée par les secrétariats et les praticiens.

Existant. Tout est hébergé sur **un seul serveur** dans les locaux : un serveur web (Nginx + application Node.js), une base **PostgreSQL** installée sur la même machine, et les documents patients stockés dans un **dossier local**. Les sauvegardes sont faites « quand on y pense » sur un disque USB. Un **compte administrateur unique** est partagé par les deux personnes de l'IT. Aucune supervision. Le serveur a déjà connu une panne de 6 heures le mois dernier.

Demande de la direction. Migrer vers **Microsoft Azure** une cible qui améliore la disponibilité, sépare les couches, **protège les données de santé** (données sensibles), maîtrise les coûts, et puisse être **redéployée à l'identique** pour créer un environnement de test. La DSI attend une **note de recommandations**.

Partie 1 — Analyse de l'existant & architecture cible (10 pts)

Q1. (3 pts) Identifiez **trois risques majeurs** de l'architecture actuelle, dans trois domaines différents, et précisez l'impact métier de chacun.

Q2. (4 pts) Proposez une **architecture cible Azure** (décrivez les composants et les flux, un schéma textuel suffit). Elle doit séparer les couches et supporter la montée en charge. Indiquez clairement **ce qui est exposé à Internet** et **ce qui est interne**.

Q3. (3 pts) Pour **chaque** service que vous proposez, indiquez s'il relève de **IaaS, PaaS ou gouvernance**, et expliquez en quoi cela change la responsabilité de l'équipe IT.

Partie 2 — Choix techniques justifiés (10 pts)

Q4. (2 pts) Pour héberger l'application web, vous hésitez entre **deux VM** et **Azure App Service**. Donnez un argument **pour chaque** option et indiquez votre choix pour MediTrack en le justifiant.

Q5. (2 pts) **Azure Load Balancer** ou **Application Gateway** devant les serveurs web ? Justifiez en vous appuyant sur la nature des données et des flux de MediTrack.

Q6. (2 pts) La base PostgreSQL doit-elle rester **sur une VM** ou passer en service **managé** ? Donnez deux critères de décision et tranchez.

Q7. (2 pts) Proposez un **plan d'adressage et de segmentation réseau** (VNet + subnets) et indiquez quelles **règles NSG** vous mettez sur chaque couche (ports + sources).

Q8. (2 pts) MediTrack manipule des **données de santé**. Citez **trois mesures** spécifiques (réseau, stockage, accès) pour protéger ces données dans l'architecture cible.

Partie 3 — Industrialisation avec Terraform (8 pts)

Q9. (2 pts) La direction veut pouvoir recréer un environnement de test **identique**. Expliquez pourquoi **Terraform** répond à ce besoin mieux que des clics dans le portail.

Q10. (2 pts) Quelle est la différence entre `terraform plan` et `terraform apply` ? Pourquoi le `plan` est-il indispensable ?

Q11. (2 pts) Un collègue veut committer le fichier `terraform.tfstate` dans le dépôt Git de l'équipe. Quels **deux problèmes** cela pose-t-il ? Que proposez-vous à la place ?

Q12. (2 pts) Après un `terraform plan`, vous voyez la ligne : `-/+ azurerm_linux_virtual_machine.web[0]`. Que signifie ce symbole et quel **risque** cela représente-t-il en production ?

Partie 4 — Exploitation, monitoring & FinOps (8 pts)

Q13. (2 pts) Citez **trois métriques** à surveiller sur les VM web de MediTrack et, pour l'une d'elles, proposez une **alerte** (seuil + ce qu'on fait quand elle se déclenche).

Q14. (2 pts) Un membre de l'équipe « arrête » les VM de test le soir via le système d'exploitation, mais la facture ne baisse pas. Expliquez pourquoi et donnez la **bonne commande/approche**.

Q15. (2 pts) La facture Azure a augmenté de 40 % ce mois-ci sans qu'on sache pourquoi. Citez **trois mesures** de gouvernance/FinOps qui auraient permis de l'éviter ou de l'expliquer rapidement.

Q16. (2 pts) Différenciez **SLI, SLO et SLA** avec un exemple chacun, dans le contexte de MediTrack.

Partie 5 — Synthèse / Note DSI (4 pts)

Q17. (4 pts) Rédigez (en ~10 lignes) la **trame** d'une note de recommandations pour la DSI de MediTrack : indiquez les rubriques attendues et, pour **trois** d'entre elles, une recommandation **priorisée et justifiée**.

Corrigé indicatif

Les réponses ci-dessous sont des **attendus** : d'autres formulations sont acceptables si elles sont justifiées. Les points clés à faire apparaître sont en gras.

Partie 1

Q1 — Risques (3 domaines distincts). - **Disponibilité** : serveur unique = **point de défaillance unique (SPOF)** → toute panne (cf. les 6 h du mois dernier) stoppe rdv, dossiers et facturation → perte d'activité et de confiance. - **Sécurité / données : compte admin partagé** (pas de traçabilité, un secret compromis = accès total) + documents patients en local non chiffrés → risque de **fuite de données de santé** (enjeu RGPD/sensibilité fort). - **Sauvegarde** : sauvegardes manuelles/irrégulières sur USB → **perte de données** probable, RPO/RTO non garantis. (*Autres acceptés : performance — web + base sur la même machine ; exploitation — aucune supervision.*)

Q2 — Architecture cible. Élément attendu : - 1 **Resource Group** dans une région (ex. France Central / Sweden Central), 1 **VNet** segmenté. - **Subnet web** : 2 **VM** (ou App Service) derrière un **Load Balancer — exposé Internet (80/443)**. - **Subnet data : base managée (Azure Database for PostgreSQL) — interne**, non exposée. - **Storage Account privé** (Blob) pour les documents — **interne**. - **NSG** par subnet, **Azure Monitor**, **Entra ID + RBAC**, **tags**. - Exposé Internet = **uniquement le Load Balancer** (+ SSH restreint à l'IP admin). Internes = base, storage, subnet data.

Q3 — Qualification. VM/VNet/NSG/LB = **IaaS** (l'équipe gère OS, patches, app). PostgreSQL managé / Storage = **PaaS** (Azure gère plateforme, sauvegardes, HA → responsabilité réduite). Entra ID/RBAC, Monitor, Cost Management = **gouvernance/supervision** (transverse). *Plus on va vers le PaaS, plus la responsabilité opérationnelle bascule vers Azure.*

Partie 2

Q4 — VM vs App Service. VM : **contrôle total**, migration proche de l'existant (Node.js déjà packagé). App Service : **moins d'administration** (pas d'OS à patcher), idéal pour une app web standard. *Choix défendable dans les deux sens* ; pour réduire l'exploitation d'une PME avec une petite équipe IT, **App Service** est un bon choix (sinon 2 VM + LB pour rester proche de l'existant).

Q5 — LB vs App Gateway. Données de santé + web exposé → besoin de **HTTPS/terminaison TLS** et idéalement d'un **WAF** → **Application Gateway (couche 7)** est le plus adapté en cible. Le Load Balancer (couche 4) suffit pour une 1re version simple mais n'offre ni WAF ni routage applicatif.

Q6 — Base managée ? Critères : **administration** (patches/sauvegardes), **disponibilité/HA**, sécurité, coût. Pour MediTrack (petite équipe, données critiques) → **base managée (Azure Database for PostgreSQL)** : sauvegardes automatiques, HA, patches gérés → décharge l'équipe et fiabilise. La base sur VM ne se justifie que si on a besoin d'un contrôle très fin du moteur.

Q7 — Réseau. Ex. VNet **10.20.0.0/16** ; **snet-web 10.20.1.0/24**, **snet-data 10.20.2.0/24**, **snet-admin 10.20.3.0/24**. - **nsg-web : 80/443** ← **Internet, 22** ← **IP admin /32**. - **nsg-data : 5432 (PostgreSQL)** ← **subnet web uniquement**, rien depuis Internet.

Q8 — Données de santé (3 mesures). Ex. : (réseau) base **non exposée** + **Private Endpoint** ; (stockage) conteneurs **privés** + **chiffrement** au repos + TLS 1.2 ; (accès) **RBAC moindre privilège** + comptes nominatifs + **MFA** + journalisation (audit). (*Bonus : Key Vault pour les secrets, Defender for Cloud.*)

Partie 3

Q9 — Terraform. Le code décrit l'**état cible** : il est **versionné, relu, rejoué** → on recrée un env de test **identique** en une commande, sans dépendre de la mémoire d'un admin ni risquer des écarts. Reproductibilité + traçabilité + destruction propre.

Q10 — plan vs apply. **plan** **prévisualise** les changements (créer/modifier/détruire) **sans rien appliquer** ; **apply** les **exécute**. Le **plan** est indispensable pour **vérifier avant d'impacter Azure** (éviter une destruction non voulue, valider noms/régions/coûts).

Q11 — tfstate dans Git. Problèmes : (1) il peut contenir des **secrets** / données sensibles → fuite ; (2) pas de **verrouillage** → conflits/corruption si plusieurs personnes ; (perte de fichier aussi). Solution : **backend distant** (Storage Account Azure + container dédié, verrouillage + RBAC) et **.gitignore** sur ***.tfstate**.

Q12 — -/+. La VM va être **remplacée : détruite puis recréée**. Risque : **interruption de service** et **perte des données locales** non externalisées. → lire le plan, comprendre ce qui force le remplacement, planifier en heures creuses.

Partie 4

Q13 — Métriques + alerte. Métriques : **CPU**, mémoire, disque, **disponibilité HTTP**, réseau. Ex. d'alerte : **CPU moyen > 80 % pendant 5 min** → notifier l'équipe (action group) et investiguer/scaler ; la procédure (runbook) doit dire quoi vérifier.

Q14 — stop vs deallocate. **stop** (ou arrêt OS) **laisse la capacité de calcul réservée** → **le compute reste facturé**. Il faut **az vm deallocate** (ou un auto-shutdown planifié), qui **libère le compute** et coupe ce coût (les disques restent facturés).

Q15 — FinOps/gouvernance (3 mesures). **Tags obligatoires** (Owner/CostCenter/Environment) pour ventiler les coûts ; **Budgets + cost alerts** pour être prévenu d'un dépassement ; **Cost Management / Advisor** pour analyser et repérer ressources orphelines/surdimensionnées. (*Azure Policy pour imposer les tags = bonus.*)

Q16 — SLI/SLO/SLA. **SLI** = indicateur **mesuré** (ex. % de requêtes < 500 ms ou taux de dispo réel). **SLO** = objectif **interne** (ex. 99,5 % de disponibilité/mois). **SLA** = engagement **contractuel** envers les cabinets clients (ex. 99 % avec pénalité). *Relation : SLI mesure, SLO vise, SLA engage.*

Partie 5

Q17 — Trame note DSI. Rubriques : **1) Contexte · 2) Constats · 3) Risques · 4) Recommandations prioritaires · 5) Plan d'action (court/moyen/long) · 6) Indicateurs de suivi**. Trois recos justifiées, ex. : - **(Haute)** Restreindre les accès admin + comptes nominatifs + MFA → réduit le risque de compromission des données de santé. - **(Haute)** Base managée + storage privé chiffré + Private Endpoint → protège les données et fiabilise les sauvegardes. - **(Moyenne)** Tags obligatoires + budget par environnement + supervision (Monitor) → maîtrise des coûts et détection des incidents.

Grille d'auto-évaluation rapide

Tu sais...	✓
Identifier un SPOF et le corriger (instances + LB)	
Qualifier IaaS / PaaS / gouvernance et la responsabilité associée	
Choisir LB (L4) vs App Gateway (L7) avec justification	
Segmenter un réseau + écrire les règles NSG (ports/sources)	
Expliquer plan/apply, state, drift, -/+	
Différencier stop et deallocate (coût)	
Distinguer métriques/logs et SLI/SLO/SLA	

Tu sais... 	
Citer 3 mesures FinOps et 3 mesures de sécurité	
Structurer une note DSI priorisée	

Réflexe d'examen : dans une mise en situation, structure toujours ta réponse en **constat** → **choix** → **justification (coût / sécurité / disponibilité / exploitation)**. C'est exactement la grille du Well-Architected Framework.